

Design by Contract

Bertrand Meyer apresenta em "Design by Contract" uma visão que, assim como o roadmap de Garlan, transcende a técnica para alcançar o filosófico, fundamentando a construção de software na confiabilidade e na clareza. Ele parte de uma metáfora poderosa: a construção de software como uma sucessão de decisões contratuais documentadas. Para Meyer, um sistema não é apenas um amontoado de código, mas um conjunto de serviços regidos por contratos precisos que protegem tanto quem solicita (o cliente) quanto quem executa (o fornecedor). Essa perspectiva transforma o design em um exercício estratégico de definição de obrigações e benefícios mútuos.

O coração desta abordagem reside nas assertivas: pré-condições, pós-condições e invariantes de classe. Meyer argumenta que, para produzir software robusto, as obrigações de cada chamada devem ser explícitas. Uma pré-condição estabelece o que deve ser verdade para que uma rotina seja executada corretamente, enquanto a pós-condição garante o resultado entregue ao final. Já o invariante de classe atua como uma cláusula geral, uma restrição de integridade que deve ser mantida em todos os estados "observáveis" do objeto. É uma visão rigorosa que busca substituir o caos das suposições informais pela precisão matemática das especificações.

Um dos pontos mais provocativos do texto é o ataque frontal à "programação defensiva". Meyer defende que o excesso de verificações redundantes gera sistemas complexos, pesados e, ironicamente, menos confiáveis. Se um contrato define que a responsabilidade por uma condição é do cliente, o fornecedor não deve testá-la novamente. Essa divisão clara de trabalho permite que as rotinas se concentrem em realizar bem uma tarefa específica, em vez de tentarem, sem sucesso, lidar com todos os casos anormais possíveis. É uma lição de economia de design: a simplicidade arquitetural é alcançada quando cada parte sabe exatamente o que lhe cabe.

Ao discutir herança e ligação dinâmica, Meyer utiliza a teoria dos contratos para iluminar o que ele chama de "subcontratação honesta". Ele estabelece uma regra fundamental para a redefinição de rotinas: um subcontratado (subclasse) pode ser mais tolerante em suas exigências (enfraquecer a pré-condição) ou mais generoso em seus resultados (fortalecer a pós-condição), mas nunca o contrário. Essa visão protege o polimorfismo, garantindo que um cliente possa substituir um fornecedor por um descendente sem que o contrato original seja quebrado. Meyer expõe os perigos da ligação estática, tratando-a como um erro que pode levar a objetos inconsistentes que violam seus próprios invariantes.

A abordagem de Meyer para exceções também é revolucionária em sua simplicidade. Ele define falha como a incapacidade de cumprir um contrato e uma exceção como o evento que sinaliza essa falha. Diferente de linguagens como Ada ou CLU, onde exceções podem se tornar saltos de controle confusos, Meyer propõe apenas duas reações legítimas: a retomada (retry), onde se tenta uma nova estratégia para cumprir o contrato, ou o "pânico organizado", onde a rotina limpa seu estado, restaura o invariante e admite o

fracasso para seu chamador. Não há meio-termo; ou o contrato é cumprido, ou a falha é notificada.

Embora Meyer admita que o monitoramento de assertivas em tempo de execução seja, idealmente, uma ferramenta de depuração enquanto não se pode provar formalmente a correção de todos os programas, ele reconhece seu valor pragmático na garantia da qualidade. O valor duradouro de "Design by Contract" não está apenas na linguagem Eiffel, mas na defesa da honestidade intelectual no desenvolvimento. Ele nos lembra que a robustez não nasce de milagres ou de código redundante, mas de componentes que declaram claramente o que farão e o que esperam em troca, aceitando a natureza parcial das funções como um fato da vida e da programação.